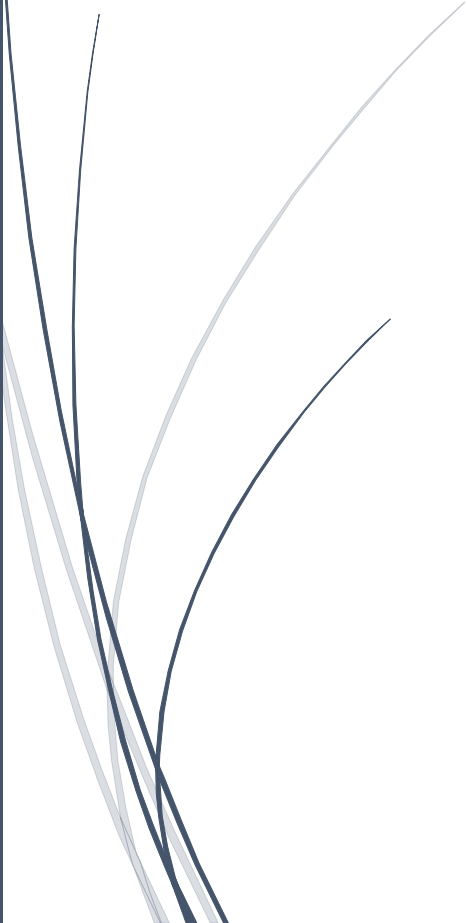


A dark blue vertical bar on the left side of the page. A blue arrow points to the right from the bar, containing the date.

11-1-2026

# NIDS

Práctica 3-2

Several thin, curved lines in dark blue and light grey originate from the bottom left and curve upwards and to the right.

SAVU, RALUCA ALEXANDRA  
CIPFP

## Tabla de contenido

|      |                                                        |   |
|------|--------------------------------------------------------|---|
| 1.   | Instalación (troubleshooting).....                     | 2 |
| 2.   | Análisis.....                                          | 3 |
| 2.1. | Configuración inicial de red.....                      | 3 |
| 2.2. | Verificación de la configuración de SNORT .....        | 3 |
| 2.3. | Ejecución de SNORT en modo IDS .....                   | 4 |
| 2.4. | Creación y análisis de las reglas personalizadas ..... | 4 |
| 2.5. | Prueba de funcionamiento mediante escaneo Nmap .....   | 6 |
| 2.6. | Detección del ataque por parte de SNORT .....          | 6 |
| 2.7. | Conclusión .....                                       | 7 |

## 1. Instalación (troubleshooting)

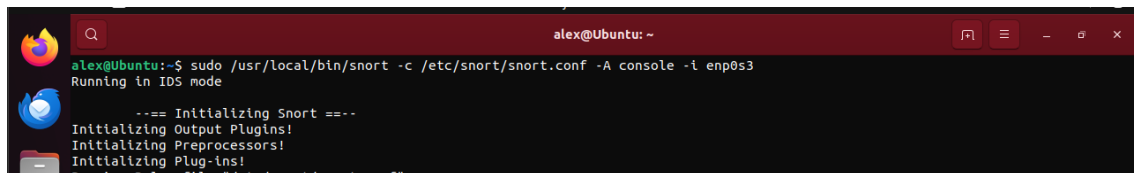
Para la práctica procedí a instalar SNORT2 descargándolo directamente desde la página oficial, ya que la instalación mediante 'apt no funcionó correctamente en mi sistema (Ubuntu 22.04). Al compilar SNORT desde el código fuente me encontré con varios problemas: por ejemplo, la configuración del SNORT la tuve que hacer con la ayuda del fichero snort.conf ya que con el comando 'dpkg-reconfigure snort' no me ha funcionado. Además, la configuración del modo promiscuo no podía añadirse directamente en snort.conf, por lo que tuve que activarlo desde la terminal utilizando el parámetro '-p' al ejecutar el IDS.

```
sudo /usr/local/bin/snort -c /etc/snort/snort.conf -i enp0s3 -v -p
```

Además, fue necesario descargar y cargar manualmente algunas reglas básicas, como 'local.rules' y 'community.rules', para que el sistema pudiera arrancar sin errores. Inicialmente, SNORT presentaba fallos al procesar ciertas reglas debido a que la directiva 'flow:stateless' no es compatible con la versión 2.9.20 que instalé; por ello, tuve que eliminarla y ajustar cada regla a una sola línea para que se ejecutara correctamente.

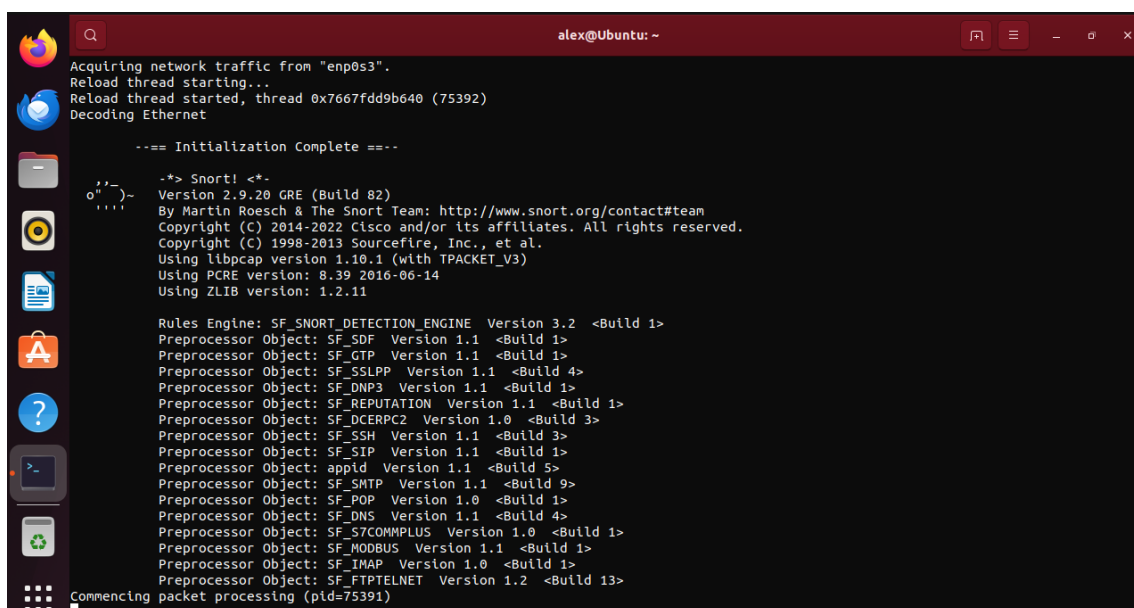
Finalmente, pude comprobar que SNORT capturaba tráfico utilizando el comando:

```
sudo /usr/local/bin/snort -c /etc/snort/snort.conf -A console -i enp0s3
```



```
alex@Ubuntu: ~
alex@Ubuntu:~$ sudo /usr/local/bin/snort -c /etc/snort/snort.conf -A console -i enp0s3
Running in IDS mode

---= Initializing Snort ---
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file "/etc/snort/snort.conf"
```



```
alex@Ubuntu: ~
Acquiring network traffic from "enp0s3".
Reload thread starting...
Reload thread started, thread 0x7667fdd9b640 (75392)
Decoding Ethernet

---= Initialization Complete ---

-*> Snort! <*-
Version 2.9.20 GRE (Build 82)
By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using libpcap version 1.10.1 (with TPACKET_V3)
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.2.11

Rules Engine: SF_SNORT_DETECTION_ENGINE Version 3.2 <Build 1>
Preprocessor Object: SF_SDF Version 1.1 <Build 1>
Preprocessor Object: SF_GTP Version 1.1 <Build 1>
Preprocessor Object: SF_SSLPP Version 1.1 <Build 4>
Preprocessor Object: SF_DNP3 Version 1.1 <Build 1>
Preprocessor Object: SF_REPUTATION Version 1.1 <Build 1>
Preprocessor Object: SF_DCERPC2 Version 1.0 <Build 3>
Preprocessor Object: SF_SSH Version 1.1 <Build 3>
Preprocessor Object: SF_SIP Version 1.1 <Build 1>
Preprocessor Object: appid Version 1.1 <Build 5>
Preprocessor Object: SF_SMTP Version 1.1 <Build 9>
Preprocessor Object: SF_POP Version 1.0 <Build 1>
Preprocessor Object: SF_DNS Version 1.1 <Build 4>
Preprocessor Object: SF_S7COMPLUS Version 1.0 <Build 1>
Preprocessor Object: SF_MODBUS Version 1.1 <Build 1>
Preprocessor Object: SF_IMAP Version 1.0 <Build 1>
Preprocessor Object: SF_FTPTELNET Version 1.2 <Build 13>
Commencing packet processing (pid=75391)
```

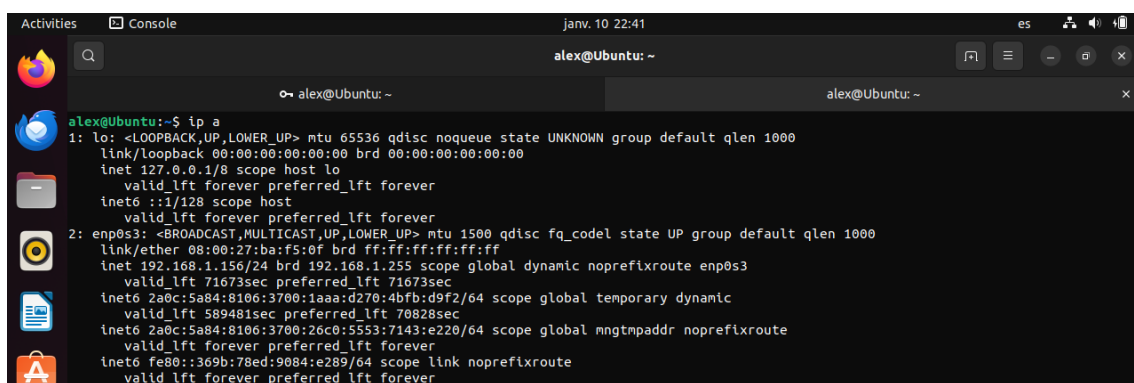
## 2. Análisis

En esta práctica se ha instalado y configurado el sistema de detección de intrusiones SNORT sobre una máquina virtual Ubuntu 22.04, con el objetivo de detectar escaneos de puertos realizados con Zenmap desde nuestra máquina real.

### 2.1. Configuración inicial de red

Antes de configurar SNORT, se identifica la interfaz de red activa y la dirección IP asignada a la máquina virtual mediante el comando 'ip a'.

La interfaz para la escucha de tráfico fue la 'enp0s3', perteneciente a la red 192.168.1.156/24, que posteriormente se configuró como red protegida en SNORT.



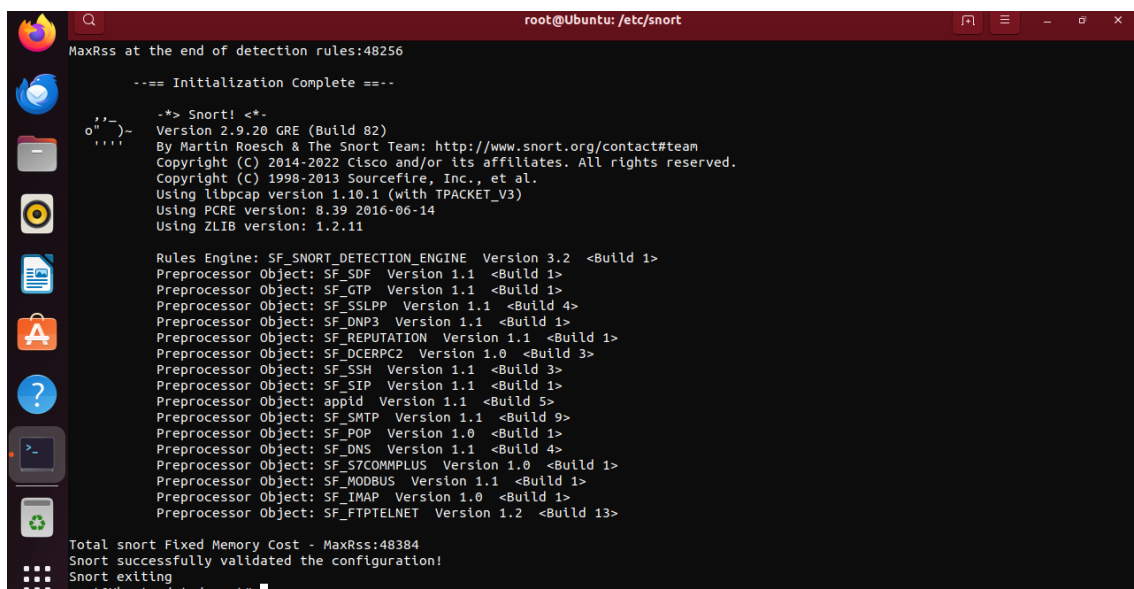
```
alex@Ubuntu:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:ba:f5:0f brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.156/24 brd 192.168.1.255 scope global dynamic noprefroute enp0s3
        valid_lft 71673sec preferred_lft 71673sec
    inet6 2a0c:5a84:8106:3700:1aaa:d270:4bfb:d9f2/64 scope global temporary dynamic
        valid_lft 589481sec preferred_lft 70828sec
    inet6 2a0c:5a84:8106:3700:26c0:5553:7143:e220/64 scope global mngtmpaddr noprefroute
        valid_lft forever preferred_lft forever
    inet6 fe80::369b:78ed:9084:e289/64 scope link noprefroute
        valid_lft forever preferred_lft forever
```

### 2.2. Verificación de la configuración de SNORT

Una vez configurado SNORT y añadidas las reglas necesarias, se comprobó que la configuración era correcta utilizando el siguiente comando loggeados con usuario root:

`snort -T -c /etc/snort/snort.conf`

El sistema validó correctamente la configuración y las reglas cargadas.



```
root@Ubuntu: /etc/snort
MaxRss at the end of detection rules:48256

==== Initialization Complete ====

-*~ Snort! <*-
Version 2.9.20 GRE (Build 82)
By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using libpcap version 1.10.1 (with TPACKET_V3)
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.2.11

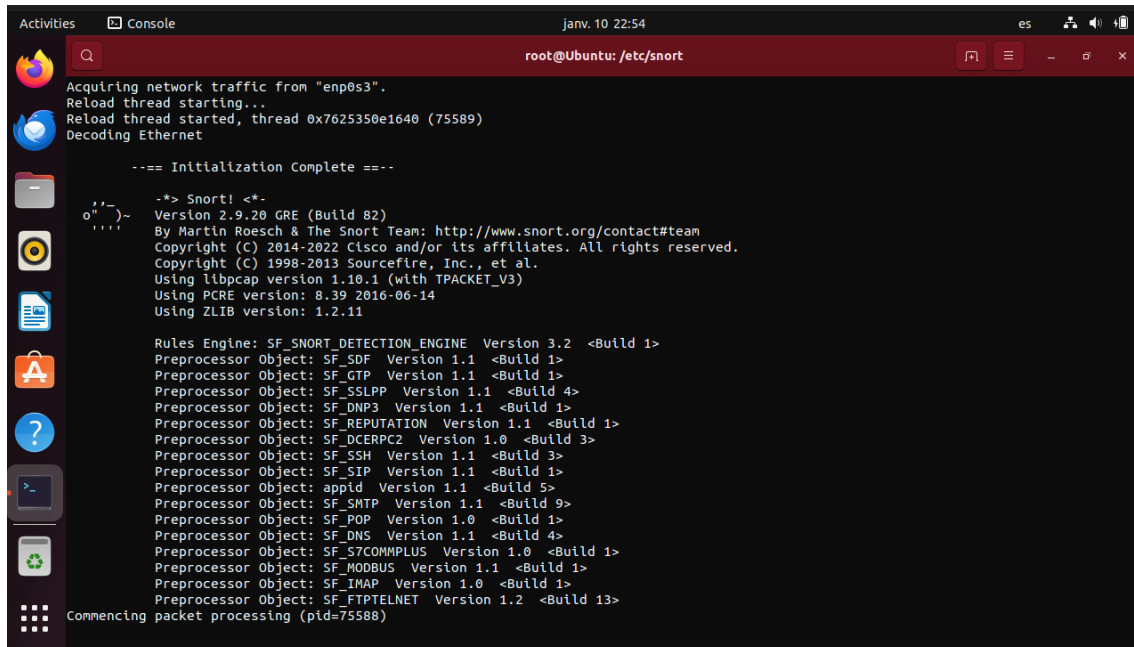
Rules Engine: SF_SNORT_DETECTION_ENGINE Version 3.2 <Build 1>
Preprocessor Object: SF_SDF Version 1.1 <Build 1>
Preprocessor Object: SF_GTP Version 1.1 <Build 1>
Preprocessor Object: SF_SSLPP Version 1.1 <Build 4>
Preprocessor Object: SF_DNP3 Version 1.1 <Build 1>
Preprocessor Object: SF_REPUTATION Version 1.1 <Build 1>
Preprocessor Object: SF_DCERPC2 Version 1.0 <Build 3>
Preprocessor Object: SF_SSH Version 1.1 <Build 3>
Preprocessor Object: SF_SIP Version 1.1 <Build 1>
Preprocessor Object: appid Version 1.1 <Build 5>
Preprocessor Object: SF_SMTP Version 1.1 <Build 9>
Preprocessor Object: SF_POP Version 1.0 <Build 1>
Preprocessor Object: SF_DNS Version 1.1 <Build 4>
Preprocessor Object: SF_S7COMMPLUS Version 1.0 <Build 1>
Preprocessor Object: SF_MODBUS Version 1.1 <Build 1>
Preprocessor Object: SF_IMAP Version 1.0 <Build 1>
Preprocessor Object: SF_FTPTELNET Version 1.2 <Build 13>

Total snort Fixed Memory Cost - MaxRss:48384
Snort successfully validated the configuration!
Snort exiting
root@Ubuntu: /etc/snort#
```

## 2.3. Ejecución de SNORT en modo IDS

Posteriormente, SNORT se ejecutó en modo consola para que quedara a la escucha del tráfico de red en la interfaz seleccionada.

```
sudo /usr/local/bin/snort -c /etc/snort/snort.conf -A console -i enp0s3
```



```

Activities Console
root@Ubuntu: /etc/snort
Acquiring network traffic from "enp0s3".
Reload thread starting...
Reload thread started, thread 0x7625350e1640 (75589)
Decoding Ethernet

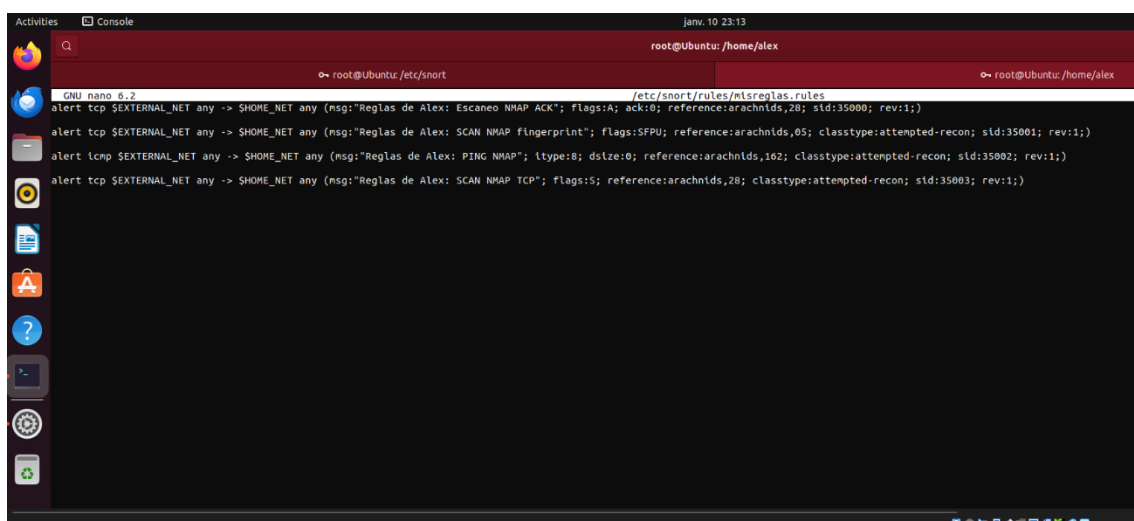
--- Initialization Complete ---

--> Snort! <--
Version 2.9.20 GRE (Build 82)
By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using libpcap version 1.10.1 (with TPACKET_V3)
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.2.11

Rules Engine: SF_SNORT DETECTION_ENGINE Version 3.2 <Build 1>
Preprocessor Object: SF_SDF Version 1.1 <Build 1>
Preprocessor Object: SF_GTP Version 1.1 <Build 1>
Preprocessor Object: SF_SSLPP Version 1.1 <Build 4>
Preprocessor Object: SF_DNP3 Version 1.1 <Build 1>
Preprocessor Object: SF_REPUTATION Version 1.1 <Build 1>
Preprocessor Object: SF_DCERPC2 Version 1.0 <Build 3>
Preprocessor Object: SF_SSH Version 1.1 <Build 3>
Preprocessor Object: SF_SIP Version 1.1 <Build 1>
Preprocessor Object: appid Version 1.1 <Build 5>
Preprocessor Object: SF_SMTP Version 1.1 <Build 9>
Preprocessor Object: SF_POP Version 1.0 <Build 1>
Preprocessor Object: SF_DNS Version 1.1 <Build 4>
Preprocessor Object: SF_S7COMPLUS Version 1.0 <Build 1>
Preprocessor Object: SF_MODBUS Version 1.1 <Build 1>
Preprocessor Object: SF_IMAP Version 1.0 <Build 1>
Preprocessor Object: SF_FTPTELNET Version 1.2 <Build 13>
Commencing packet processing (pid=75588)
  
```

## 2.4. Creación y análisis de las reglas personalizadas

Se creó un fichero de reglas personalizadas llamado 'misreglas.rules', ubicado en el directorio '/etc/snort/rules', con el objetivo de detectar diferentes técnicas de escaneo utilizadas por Zenmap.



```

Activities Console
root@Ubuntu: /home/alex
GNU nano 6.2 /etc/snort/rules/misreglas.rules
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"Reglas de Alex: Escaneo NMAP ACK"; flags:A; ack:0; reference:arachnids,28; sid:35000; rev:1;)
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"Reglas de Alex: SCAN NMAP fingerprint"; flags:SFP; reference:arachnids,05; classtype:attempted-recon; sid:35001; rev:1;)
alert icmp $EXTERNAL_NET any -> $HOME_NET any (msg:"Reglas de Alex: PING NMAP"; ltype:8; dsize:0; reference:arachnids,102; classtype:attempted-recon; sid:35002; rev:1;)
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"Reglas de Alex: SCAN NMAP TCP"; flags:S; reference:arachnids,28; classtype:attempted-recon; sid:35003; rev:1;)
  
```

**\*\*Tuve que añadir las reglas en una sola línea para que SNORT lo reconociera.**

- **Regla 1: Detección de escaneo ACK**

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"Reglas de Alex: Escaneo NMAP ACK"; flags:A; ack:0; reference:arachnids,28; sid:35000; rev:1;)
```

Esta regla detecta paquetes TCP con el flag ACK activado y número de acuse (ack) a cero, una técnica típica utilizada por Nmap para identificar reglas de firewall y hosts activos.

Cualquier paquete con estas características que llegue desde una red externa hacia la red protegida generará una alerta.

- **Regla 2: Detección de fingerprinting de Nmap**

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"Reglas de Alex: SCAN NMAP fingerprint"; flags:SFP; reference:arachnids,05; classtype:attempted-recon; sid:35001; rev:1;)
```

Esta regla detecta combinaciones anómalas de flags TCP (S, F, P y U), muy poco habituales en tráfico legítimo y utilizadas por Nmap para fingerprinting del sistema remoto.

- **Reglas 3: Detección de ping ICMP de Nmap**

```
alert icmp $EXTERNAL_NET any -> $HOME_NET any (msg:"Reglas de Alex: PING NMAP"; itype:8; dsize:0; reference:arachnids,162; classtype:attempted-recon; sid:35002; rev:1;)
```

Esta regla detecta paquetes ICMP de tipo echo request (ping) sin carga útil, utilizados por Nmap para descubrir hosts activos en la red.

- **Regla 4: Detección genérica de escaneo TCP**

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"Reglas de Alex: SCAN NMAP TCP"; flags:S; reference:arachnids,28; classtype:attempted-recon; sid:35003; rev:1;)
```

Esta regla actúa como una detección genérica de escaneos TCP hacia la red protegida. Permite detectar intentos de reconocimiento incluso cuando no se utilizan técnicas específicas de flags.

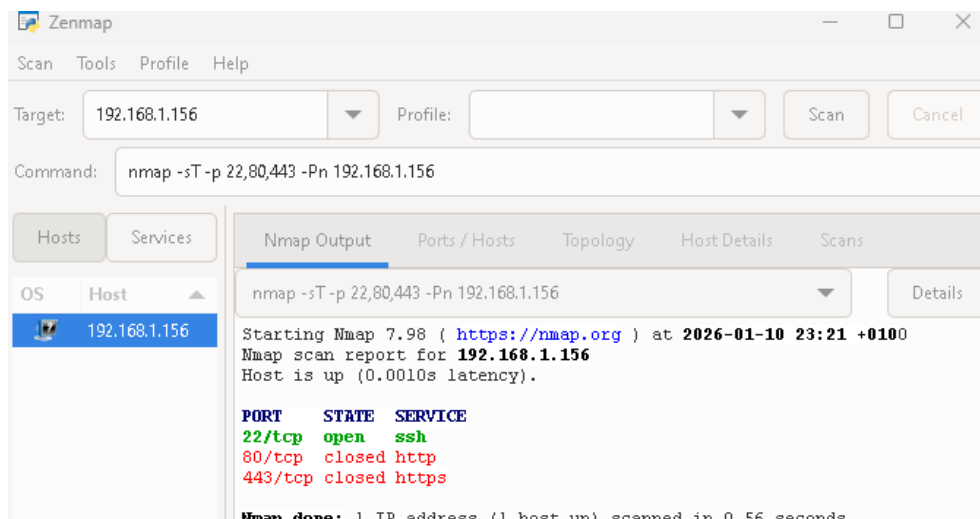
\*\* El fichero de reglas personalizado fue añadido correctamente al fichero de configuración principal de SNORT mediante la línea:

```
include $RULE_PATH/misreglas.rules
```

## 2.5. Prueba de funcionamiento mediante escaneo Nmap

Para comprobar el correcto funcionamiento del IDS, se realizó un escaneo de puertos desde una máquina Windows utilizando Zenmap, apuntando a la dirección IP de la máquina virtual Ubuntu:

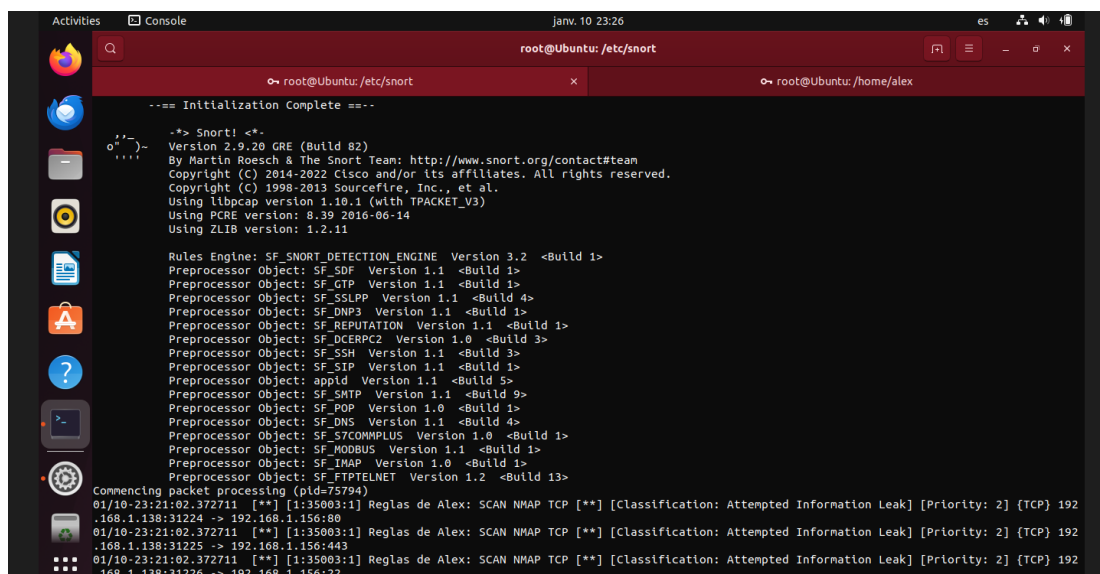
```
nmap -sT -p 22,80,443 -Pn 192.168.1.156
```



Este comando realiza: un escaneo TCP Connect (**-sT**); deshabilita la fase de descubrimiento de hosts (**-Pn**), forzando el escaneo incluso cuando el objetivo no responde a peticiones ICMP; especifica los puertos a analizar (**-p 22,80,443**).

## 2.6. Detección del ataque por parte de SNORT

Durante la ejecución del escaneo, SNORT generó las alertas correspondientes en la consola, indicando la detección de un intento de reconocimiento y escaneo de puertos contra la máquina protegida.



## 2.7. Otros escaneos

A continuación, ejecutaremos otros comandos para comprobar el funcionamiento de nuestras reglas.

### 1. Escaneo de puertos completo TCP

Detecta escaneos TCP genéricos hacia múltiples puertos.

```
nmap -sT -Pn 192.168.1.156
```

```

root@Ubuntu: /etc/snort
192.168.1.138:18219 -> 192.168.1.156:25735
01/10-23:46:22.158066 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18220 -> 192.168.1.156:1088
01/10-23:46:22.158066 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18221 -> 192.168.1.156:5915
01/10-23:46:22.158066 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18222 -> 192.168.1.156:2910
01/10-23:46:22.158066 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18223 -> 192.168.1.156:5001
01/10-23:46:22.158066 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18224 -> 192.168.1.156:407
01/10-23:46:22.158066 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18225 -> 192.168.1.156:1187
01/10-23:46:22.158066 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18226 -> 192.168.1.156:1277
01/10-23:46:22.159096 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18227 -> 192.168.1.156:2170
01/10-23:46:22.159096 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18228 -> 192.168.1.156:9000
01/10-23:46:22.159096 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18229 -> 192.168.1.156:1148
01/10-23:46:22.159096 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18230 -> 192.168.1.156:9595
01/10-23:46:22.159096 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18231 -> 192.168.1.156:9103
01/10-23:46:22.159096 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18233 -> 192.168.1.156:17877
01/10-23:46:22.159096 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18234 -> 192.168.1.156:5666
01/10-23:46:22.159096 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18235 -> 192.168.1.156:458
01/10-23:46:22.160374 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18236 -> 192.168.1.156:1060
01/10-23:46:22.160375 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18237 -> 192.168.1.156:3324

```

### 2. Escaneo rápido de los 100pueros más comunes

Escaneo rápido y realista usado en fases iniciales de ataque.

```
nmap -sT -Pn -F 192.168.1.156
```

```

root@Ubuntu: /etc/snort
192.168.1.138:18262 -> 192.168.1.156:1119
01/10-23:54:53.212507 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18263 -> 192.168.1.156:8081
01/10-23:54:53.212507 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18264 -> 192.168.1.156:1029
01/10-23:54:53.212507 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18265 -> 192.168.1.156:49156
01/10-23:54:53.213412 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18266 -> 192.168.1.156:8000
01/10-23:54:53.213412 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18267 -> 192.168.1.156:144
01/10-23:54:53.213412 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18268 -> 192.168.1.156:13
01/10-23:54:53.213412 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18269 -> 192.168.1.156:514
01/10-23:54:53.213412 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18270 -> 192.168.1.156:5800
01/10-23:54:53.213412 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18271 -> 192.168.1.156:8009
01/10-23:54:53.213412 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18272 -> 192.168.1.156:5051
01/10-23:54:53.213412 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18273 -> 192.168.1.156:465
01/10-23:54:53.214732 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18274 -> 192.168.1.156:106
01/10-23:54:53.214732 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18275 -> 192.168.1.156:2049
01/10-23:54:53.214733 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18276 -> 192.168.1.156:3986
01/10-23:54:53.214733 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18277 -> 192.168.1.156:119
01/10-23:54:53.214733 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18278 -> 192.168.1.156:515
01/10-23:54:53.214733 [**] [1:35003:1] Reglas de Alex: SCAN NMAP TCP [**] [Classification: Attempted Information Leak] [Priority: 2] {TCP} 192
192.168.1.138:18279 -> 192.168.1.156:2717

```



## 2.8. Conclusión

Gracias a la creación de reglas personalizadas y a la correcta configuración de SNORT, el sistema fue capaz de detectar de forma efectiva distintos tipos de escaneos realizados con Zenmap.

Esto demuestra la funcionalidad de SNORT como sistema de detección de intrusiones en red (NIDS) para la identificación temprana de actividades de reconocimiento y posibles ataques.